

Lab 10 — Capstone — Full Multi-Tier Architecture

Validate a complete five-tier architecture end to end, break the application tier, restore it, and map every component to its AWS service.

Lab environment: Docker Desktop (free local simulation) — runs locally, no AWS account needed

Estimated time: 35–40 minutes

SAA-C03 domain: All four SAA-C03 domains

Overview

Local component	AWS equivalent
lab10-proxy (NGINX, port 8090)	Route 53 + Application Load Balancer
lab10-app (Flask)	Amazon ECS service
lab10-db (MariaDB)	Amazon RDS
lab10-redis (Redis)	Amazon ElastiCache
lab10-minio (MinIO)	Amazon S3

Step 1 — Get the lab source and start the lab

```
git clone https://github.com/mohanpothula/aws-saa.git
cd aws-saa/labs/lab-10-capstone-multi-tier
docker compose up -d
docker compose ps
```

MariaDB needs 10–20 seconds to initialise on first start. If Step 5 fails with a connection error, wait a moment and run it again.

Step 2 — Validate the tiers

Task: Start and inspect all components.

```
docker compose up -d
docker compose ps
```

Expected result: All five tiers are running.

Step 3 — Test web and API

Task: Call the public and API endpoints.

```
curl http://localhost:8090/
curl http://localhost:8090/api/health
```

Expected result: LAB10 APP and {"status": "UP"}.

Step 4 — Validate the cache

Task: Set and retrieve a session value.

```
docker exec lab10-redis redis-cli SET session:1001 active
docker exec lab10-redis redis-cli GET session:1001
```

Expected result: active is returned.

Step 5 — Validate the database

Task: Create an order record.

```
docker exec lab10-db mariadb -uroot -pPass123! appdb -e "CREATE TABLE orders(id INT,status VARCHAR(20)); INSERT INTO orders VALUES(1001,'NEW'); SELECT * FROM orders;"
```

Expected result: Order **1001** is returned.

Step 6 — Failure challenge

Task: Stop the app, observe the failure, then restore it.

```
docker compose stop app
curl -i http://localhost:8090/api/health
docker compose start app
sleep 2
curl http://localhost:8090/api/health
```

Expected result: A **504 Gateway Time-out** within about 2 seconds while the app is stopped, then **{"status": "UP"}** once it is restored.

The proxy fails fast because **nginx.conf** sets **proxy_connect_timeout 2s**. Without it, NGINX keeps trying the stopped container's old address and the request hangs for up to a minute. An ALB behaves the same way: when no healthy target answers, it returns a 5xx error instead of waiting.

Step 7 — Architecture justification

Task: Map the local components to Route 53, ALB, ECS, RDS, ElastiCache and S3.

```
echo 'proxy=? app=? db=? redis=? minio=?'
```

Expected result: You explain the AWS equivalent of every component.

Verification

Check	Expected
Step 2 — Validate the tiers	All five tiers are running
Step 3 — Test web and API	LAB10 APP and {"status": "UP"}
Step 4 — Validate the cache	active is returned
Step 5 — Validate the database	Order 1001 is returned
Step 6 — Failure challenge	A 504 Gateway Time-out within about 2 seconds while the app is stopped, then {"status": "UP"} once it is restored
Step 7 — Architecture justification	You explain the AWS equivalent of every component

Completion check

- [] Every validation result demonstrated to the trainer
- [] You can say which AWS service each local component represents
- [] You can explain the blast radius of the failure in Step 6

Cleanup

```
docker compose down -v
```

-v also removes the lab's volumes, so the next run starts from a clean state.

Reference

- Lab source code: <https://github.com/mohanpothula/aws-saa/tree/main/labs/lab-10-capstone-multi-tier>
- Original student guide: [courseware/AWS_SAA_Lab10_Student_Guide.docx](#)
- [Amazon Route 53 Developer Guide](#)
- [What is an Application Load Balancer?](#)
- [Amazon ECS Developer Guide](#)
- [Amazon RDS User Guide](#)
- [Amazon ElastiCache](#)
- [Amazon S3 User Guide](#)
- [AWS Well-Architected Framework](#)
- [Docker Compose](#)
- [NGINX proxy module \(proxy_pass, proxy_cache\)](#)
- [MariaDB official Docker image](#)
- [MinIO container documentation](#)